

C# Methods - Complete Guide for Interviews and Development

Table of Contents

1. Introduction to Methods
 2. Method Parameters
 3. Return Types and Void Methods
 4. Method Overloading
 5. Optional Parameters and Named Arguments
 6. Static vs Instance Methods
 7. Local Functions (Nested Methods)
 8. Extension Methods
 9. Async and Await in Methods
 10. Lambda Expressions and Anonymous Methods
 11. Method Signatures and Overriding
 12. Access Modifiers in Methods
 13. Partial Methods
 14. Frequently Asked Interview Questions
-

1. Introduction to Methods

What is a Method?

A method is a block of code that performs a specific task and can be executed whenever required.

Methods help in:

- Code Reusability
- Modularity
- Readability
- Maintainability
- Reducing Code Duplication

Syntax

```
access_modifier return_type MethodName(parameters)
{
    // Method Body
}
```

Example:

```
using System;

class Program
{
    static void Greet()
    {
        Console.WriteLine("Hello World");
    }

    static void Main()
    {
        Greet();
    }
}
```

Output:

```
Hello World
```

Method Components

Component	Description
access_modifier	Controls accessibility
return_type	Type of value returned
MethodName	Name of method
parameters	Input values
method body	Logic executed

2. Method Parameters

Parameters allow data to be passed into methods.

Example:

```
static void Add(int x, int y)
{
```

```
        Console.WriteLine(x + y);  
    }
```

Calling:

```
Add(10, 20);
```

Output:

```
30
```

Types of Parameters

Value Parameters

Default behavior in C#.

```
static void Change(int x)  
{  
    x = 100;  
}  
  
static void Main()  
{  
    int num = 10;  
  
    Change(num);  
  
    Console.WriteLine(num);  
}
```

Output:

```
10
```

Reason: A copy of the value is passed.

ref Parameters

Passes variable by reference.

```
static void Change(ref int x)
{
    x = 100;
}

static void Main()
{
    int num = 10;

    Change(ref num);

    Console.WriteLine(num);
}
```

Output:

100

out Parameters

Used to return multiple values.

```
static void GetValues(out int x, out int y)
{
    x = 10;
    y = 20;
}
```

Usage:

```
GetValues(out int a, out int b);
```

in Parameters

Read-only reference parameter.

```
static void Display(in int x)
{
```

```
        Console.WriteLine(x);  
    }
```

Cannot modify x inside method.

params Keyword

Allows variable number of arguments.

```
static int Sum(params int[] values)  
{  
    int total = 0;  
  
    foreach(int value in values)  
    {  
        total += value;  
    }  
  
    return total;  
}
```

Usage:

```
Console.WriteLine(Sum(1,2,3,4,5));
```

Output:

```
15
```

3. Return Types and Void Methods

Void Methods

Return nothing.

```
static void PrintMessage()  
{
```

```
        Console.WriteLine("Welcome");  
    }
```

Usage:

```
PrintMessage();
```

Returning Values

```
static int Add(int x, int y)  
{  
    return x + y;  
}
```

Usage:

```
int result = Add(5, 6);  
  
Console.WriteLine(result);
```

Output:

```
11
```

Returning Strings

```
static string GetName()  
{  
    return "Somasekhar";  
}
```

Returning Objects

```
class Employee  
{
```

```
    public string Name;
}

static Employee GetEmployee()
{
    return new Employee
    {
        Name = "John"
    };
}
```

4. Method Overloading

Method Overloading means multiple methods having:

- Same Name
- Different Parameters

Compiler determines which method to execute.

Example:

```
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

Valid:

```
Add(10,20);  
Add(10.5,20.5);  
Add(1,2,3);
```

Cannot overload based only on return type.

Invalid:

```
int Add(int a, int b)  
double Add(int a, int b)
```

5. Optional Parameters and Named Arguments

Optional Parameters

Provide default values.

```
static void Display(string name = "Guest")  
{  
    Console.WriteLine(name);  
}
```

Usage:

```
Display();  
Display("Somasekhar");
```

Output:

```
Guest  
Somasekhar
```

Named Arguments

Specify parameter names.

```
static void PrintDetails(string name, int age)
{
    Console.WriteLine($"{name} {age}");
}
```

Usage:

```
PrintDetails(age: 25, name: "Somasekhar");
```

Order doesn't matter.

6. Static vs Instance Methods

Static Method

Belongs to class.

```
class Calculator
{
    public static int Add(int x, int y)
    {
        return x + y;
    }
}
```

Usage:

```
Calculator.Add(10,20);
```

No object required.

Instance Method

Belongs to object.

```
class Calculator
{
    public int Add(int x, int y)
```

```
{  
    return x + y;  
}
```

Usage:

```
Calculator calc = new Calculator();  
  
calc.Add(10,20);
```

Object required.

Interview Difference

Static	Instance
Belongs to class	Belongs to object
No object needed	Object needed
Memory allocated once	Per object
Cannot access instance members directly	Can access both

7. Local Functions (Nested Methods)

A method declared inside another method.

```
static void Main()  
{  
    int Add(int a, int b)  
    {  
        return a + b;  
    }  
  
    Console.WriteLine(Add(10,20));  
}
```

Benefits:

- Better encapsulation

- Improves readability
 - Used only inside parent method
-

8. Extension Methods

Allows adding methods to existing types without modifying source code.

```
public static class StringExtensions
{
    public static string ReverseText(this string text)
    {
        char[] chars = text.ToCharArray();

        Array.Reverse(chars);

        return new string(chars);
    }
}
```

Usage:

```
string name = "Somasekhar";

Console.WriteLine(name.ReverseText());
```

Output:

```
rahkesa moS
```

Keyword:

```
this
```

Required on first parameter.

9. Async and Await in Methods

Used for asynchronous programming.

```
using System.Threading.Tasks;

static async Task DownloadData()
{
    await Task.Delay(2000);

    Console.WriteLine("Completed");
}
```

Call:

```
await DownloadData();
```

Benefits:

- Non-blocking
- Better scalability
- Improved responsiveness

Common Return Types:

```
Task
Task<T>
ValueTask
```

10. Lambda Expressions and Anonymous Methods

Lambda Expression

```
Func<int,int> square = x => x * x;

Console.WriteLine(square(5));
```

Output:

```
25
```

Multi-line Lambda

```
Func<int,int,int> add =  
(a,b) =>  
{  
    return a + b;  
};
```

Anonymous Method

```
Action greet =  
delegate()  
{  
    Console.WriteLine("Hello");  
};  
  
greet();
```

Lambdas are generally preferred.

11. Method Signatures and Overriding

Method Signature

A method signature consists of:

- Method Name
- Number of Parameters
- Parameter Types
- Parameter Order

Example:

```
Add(int,int)  
Add(double,double)  
Add(int,int,int)
```

All have different signatures.

Return type is NOT part of method signature.

Method Overriding

Base Class:

```
class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal Sound");
    }
}
```

Derived Class:

```
class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Bark");
    }
}
```

Usage:

```
Animal animal = new Dog();

animal.Speak();
```

Output:

```
Bark
```

12. Access Modifiers in Methods

public

Accessible everywhere.

```
public void Show()  
{  
}
```

private

Accessible only inside class.

```
private void Show()  
{  
}
```

protected

Accessible in derived classes.

```
protected void Show()  
{  
}
```

internal

Accessible within same assembly.

```
internal void Show()  
{  
}
```

protected internal

Accessible:

- Same assembly
 - Derived classes
-

private protected

Accessible:

- Same assembly
 - Derived classes only
-

13. Partial Methods

Used within partial classes.

File 1:

```
public partial class Employee
{
    partial void Validate();
}
```

File 2:

```
public partial class Employee
{
    partial void Validate()
    {
        Console.WriteLine("Validation Complete");
    }
}
```

Common Uses:

- Source Generators
- Entity Framework
- Auto-generated code

Frequently Asked Interview Questions

1. What is a method?
2. Difference between parameters and arguments?
3. Difference between ref, out and in?
4. What is method overloading?
5. Can methods be overloaded by return type?
6. Difference between static and instance methods?
7. What are extension methods?
8. What is async and await?
9. What is method overriding?
10. What is a method signature?
11. Difference between lambda and anonymous methods?
12. What are access modifiers?
13. What are partial methods?
14. What is the purpose of params keyword?
15. Why is Main method static?